Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

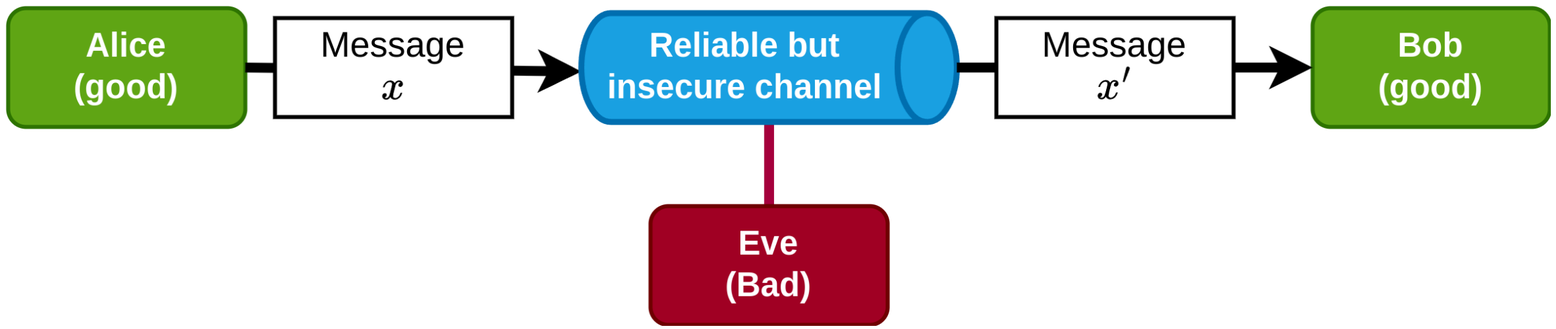
Symmetric Cryptography

Data Privacy and Security

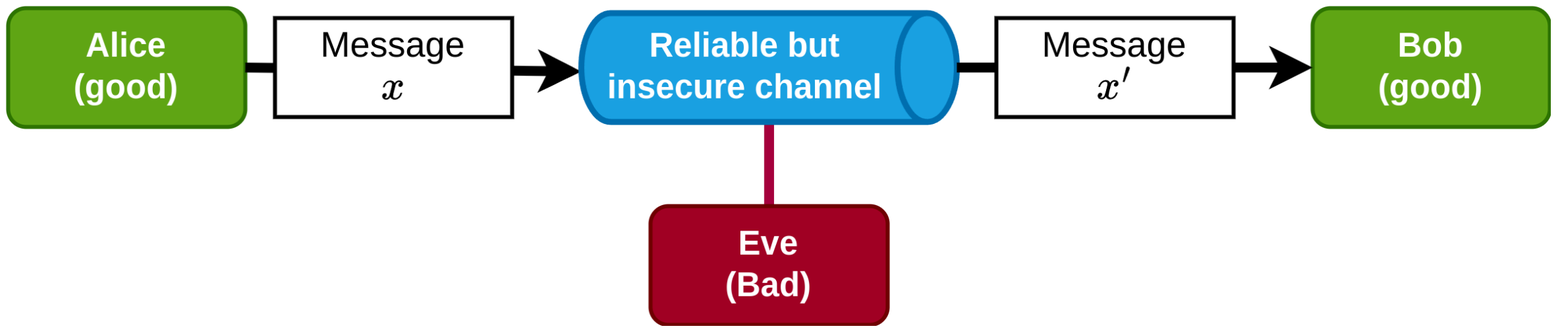
Data Science and Management (DASMA)

LUISS

Communication Model

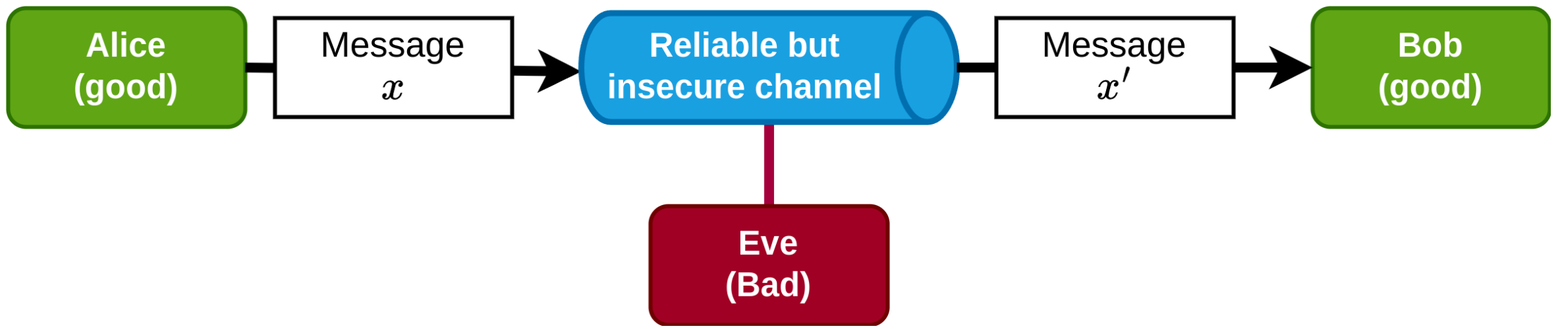


Communication Model



We do not know whether the message x is the same as the message x'.

Communication Model



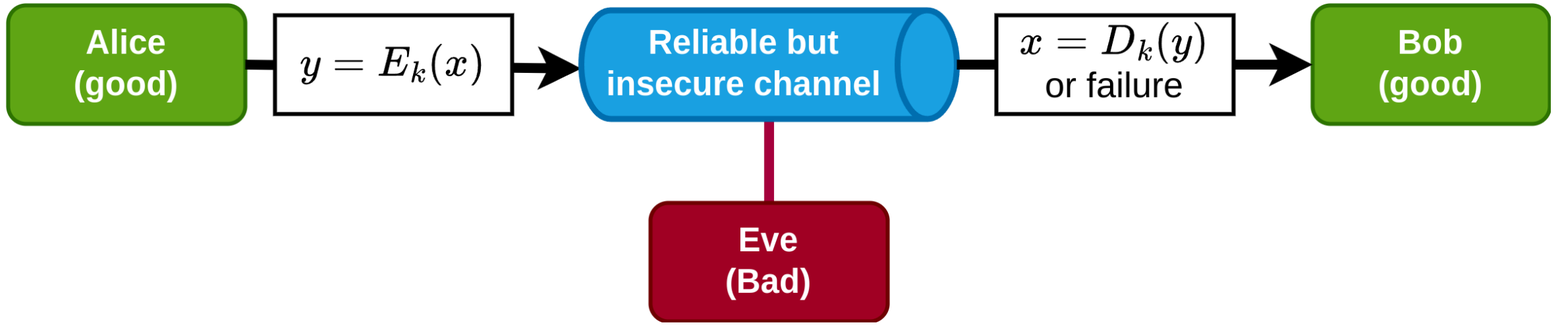
We do not know whether the message x is the same as the message x'.

How can we build a <u>secure</u> communication channel between Alice and Bob?

Secure Communication Channel using Symmetric Cipher

We use a **symmetric cipher** to:

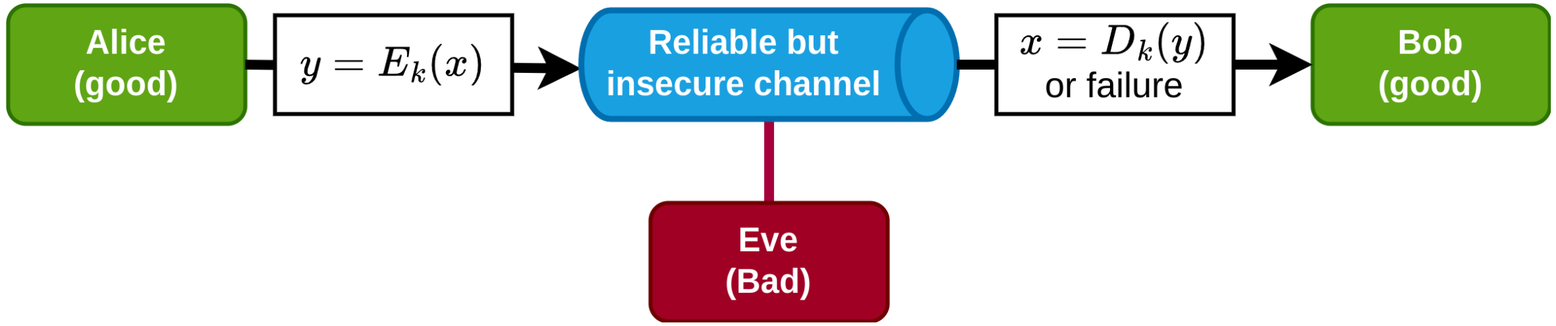
- encrypt $(E_k(x))$ using a key k to obtain the ciphertext y
- decrypt $(D_k(y))$ using the same key k the ciphertext y to obtain the plaintext x



Secure Communication Channel using Symmetric Cipher

We use a **symmetric cipher** to:

- encrypt $(E_k(x))$ using a key k to obtain the ciphertext y
- decrypt $(D_k(y))$ using the same key k the ciphertext y to obtain the plaintext x

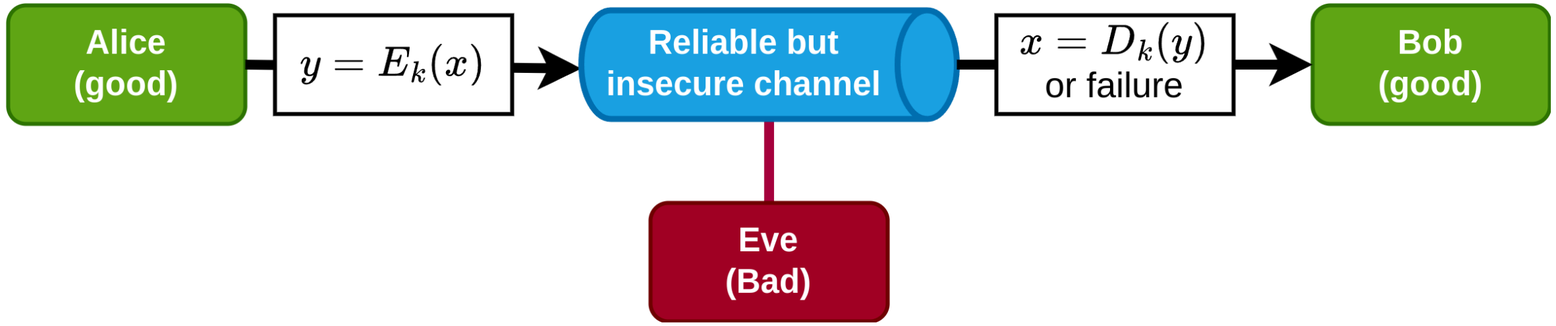


NOTE: We are assuming that the key has been securely shared between Alice and the Bob!

Secure Communication Channel using Symmetric Cipher

We use a **symmetric cipher** to:

- encrypt $(E_k(x))$ using a key k to obtain the ciphertext y
- decrypt $(D_k(y))$ using the same key k the ciphertext y to obtain the plaintext x



NOTE: We are assuming that the key has been securely shared between Alice and the Bob!

How can we implement E_k and D_k?

Example #1: Shift Cipher (Caesar's cipher)

IDEA: Shift letters in the alphabet by k position (if k=13 then is called ROT13)

$$y_i = E_k(x_i) = (x_i + k) \mod 26$$
$$x_i = D_k(y_i) = (y_i - k) \mod 26$$

Example #1: Shift Cipher (Caesar's cipher)

IDEA: Shift letters in the alphabet by k position (if k=13 then is called ROT13)

$$y_i = E_k(x_i) = (x_i + k) \mod 26$$
$$x_i = D_k(y_i) = (y_i - k) \mod 26$$

For instance, when $k = 23$:

- Plain: ABCDEFGHIJKLMNOPQRSTUVW**XYZ**
- Cipher: **XYZ**ABCDEFGHIJKLMNOPQRSTUVW

- Plaintext: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
- Ciphertext: QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD

Example #2: Substitution Cipher

IDEA: Map each letter to a different letter (k is the mapping, chosen randomly)

Example #2: Substitution Cipher

IDEA: Map each letter to a different letter (k is the mapping, chosen randomly)

For instance:

- Plain: ABCDEFGHIJKLMNOPQRSTUVWXYZ
- Cipher: ZEBRASCDFGHIJKLMNOPQTUVWXY

- Plaintext: FLEE AT ONCE. WE ARE DISCOVERED
- Ciphertext: SIAA ZQ LKBA. VA ZOA RFPBLUAOAR!

Can we break these two ciphers?

Can we break these two ciphers?

YES! Possible attacks:

Can we break these two ciphers?

YES! Possible attacks:

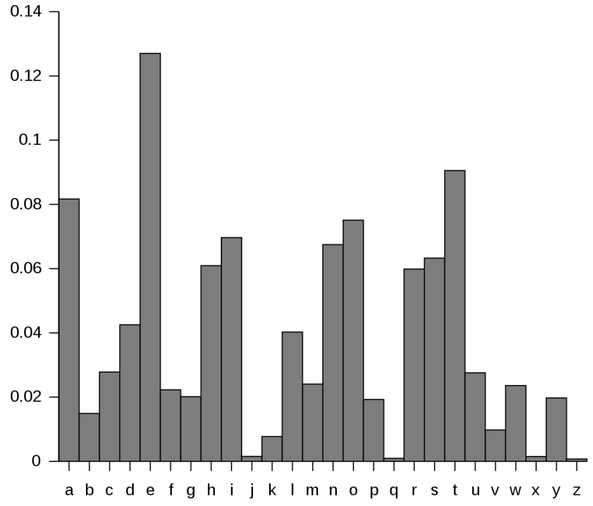
- Brute force (or exhaustive key search):
  - Shift cipher: try values of k in $[0, 26] \Longrightarrow$ trivial to break by hand
  - Substitution cipher: try all 26! mapping = 288 $\Longrightarrow$ it seems to be secure....

Can we break these two ciphers?

YES! Possible attacks:

- Brute force (or exhaustive key search):
  - Shift cipher: try values of k in $[0, 26] \Longrightarrow$ trivial to break by hand
  - Substitution cipher: try all 26! mapping = 288 $\Longrightarrow$ it seems to be secure....

- Letter Frequency Analysis:
  - we exploit that the statistical property of the text are not affected by the substitution (e.g., on average we expect the most frequent letter in the cipher text to be the letter e in the plain text). Generalize this idea by looking at pair (e.g., an) and triples (e.g., "the") to break the entire algorithm.



But why can we break these two ciphers?

Two key lessons learned:

But why can we break these two ciphers?

Two key lessons learned:

1. Key space should be large to avoid brute force.

But why can we break these two ciphers?

Two key lessons learned:

1. Key space should be large to avoid brute force.

2. Ciphers should hide the statistical properties of the encrypted plaintext.
   The ciphertext symbols should appear to be random.

Perfect cipher

A cipher over plaintext space $\{0,1\}^n$, key space $\mathcal{K}$, and ciphertext space is **perfect** if for all plaintexts x and ciphertexts y, when:

$$\Pr[x|y] = \Pr[x]$$

Hence, observing the ciphertext y gives no information about the plaintext x, i.e., ***the ciphertext does not reveal any information on the plaintext***.

Proof (intuitive)

Starting from the definition of conditional probability:

$$Pr[x|y] = \frac{Pr[x \cap y]}{Pr[y]}$$

If x and y are independent, then:

$$Pr[x \cap y] = Pr[x]Pr[y]$$

Hence:

$$Pr[x|y] = \frac{Pr[x]Pr[y]}{Pr[y]} = Pr[x]$$

(assuming $Pr[y] \neq 0$)

Background: Exclusive OR (XOR)

In computer science, a popular operation is the exclusive or (XOR) operation:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

INTUITION: only when the two inputs are different, the result is 1.

One-Time-Pad (OTP)

- Plaintext space: $\{0,1\}^n$
- Key space: $\{0,1\}^n$
- Symmetric scheme, key chosen using a True Random Number Generator (TRNG)
- Key is only used once (i.e., two messages must be encrypted with two different keys)

$$\mathrm{E}_k(x) = x \oplus k$$
$$\mathrm{D}_k(y) = y \oplus k = (x \oplus k) \oplus k = x \oplus (k \oplus k) = x \oplus 0 = x$$

OTP is a perfect cipher!

One-Time-Pad (OTP)

- Plaintext space: $\{0,1\}^n$
- Key space: $\{0,1\}^n$
- Symmetric scheme, key chosen using a True Random Number Generator (TRNG)
- Key is only used once (i.e., two messages must be encrypted with two different keys)

$$\mathrm{E}_k(x) = x \oplus k$$
$$\mathrm{D}_k(y) = y \oplus k = (x \oplus k) \oplus k = x \oplus (k \oplus k) = x \oplus 0 = x$$

OTP is a perfect cipher!

PROBLEM #1: $|k| = |x|$, i.e., the key and the plaintext must have the same length!

One-Time-Pad (OTP)

- Plaintext space: $\{0,1\}^n$
- Key space: $\{0,1\}^n$
- Symmetric scheme, key chosen using a True Random Number Generator (TRNG)
- Key is only used once (i.e., two messages must be encrypted with two different keys)

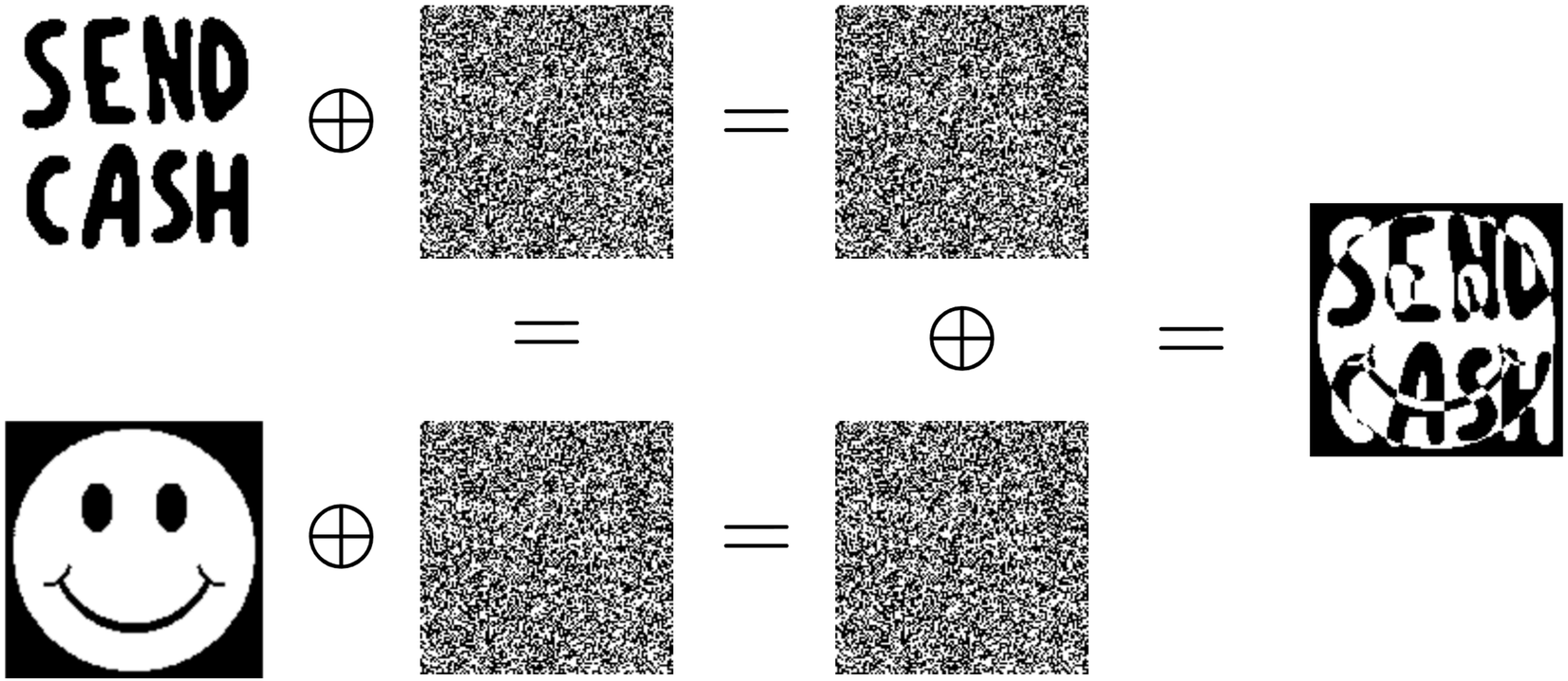
$$\mathrm{E}_k(x) = x \oplus k$$
$$\mathrm{D}_k(y) = y \oplus k = (x \oplus k) \oplus k = x \oplus (k \oplus k) = x \oplus 0 = x$$

OTP is a perfect cipher!

PROBLEM #1: $|k| = |x|$, i.e., the key and the plaintext must have the same length!

PROBLEM #2: We must change key for each message.

Why cannot we use the same keystream twice in OTP?



Can we have a perfect cipher with $|\mathbf{k}| < |\mathbf{x}|$?

Can we have a perfect cipher with $|\mathbf{k}| < |\mathbf{x}|$?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Can we have a perfect cipher with $|\mathbf{k}| < |\mathbf{x}|$?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

Can we have a perfect cipher with |k| < |x|?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

1. We assume that the keyspace is smaller than the message space: $2^{|k|} < 2^{|x|}$.

Can we have a perfect cipher with |k| < |x|?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

1. We assume that the keyspace is smaller than the message space: $2^{|k|} < 2^{|x|}$.
2. We know that the ciphertext value y_0 occurs with non-zero probability: $\Pr[y_0] > 0$.

Can we have a perfect cipher with |k| < |x|?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

1. We assume that the keyspace is smaller than the message space: $2^{|k|} < 2^{|x|}$.
2. We know that the ciphertext value y_0 occurs with non-zero probability: $\Pr[y_0] > 0$.
3. We define the set of all decryptions of y_0 under all keys: $S = \{D_k(y_0) \mid k \in \mathbb{K}\}$.

Can we have a perfect cipher with |k| < |x|?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

1. We assume that the keyspace is smaller than the message space: $2^{|k|} < 2^{|x|}$.
2. We know that the ciphertext value y_0 occurs with non-zero probability: $\Pr[y_0] > 0$.
3. We define the set of all decryptions of y_0 under all keys: $S = \{D_k(y_0) \mid k \in \mathbb{K}\}$.
4. From the size mismatch between keys and messages, it follows that there exists some message x that is not in this decryption set: $\exists k \in \mathbb{K} : x \notin S$.

Can we have a perfect cipher with |k| < |x|?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

1. We assume that the keyspace is smaller than the message space: $2^{|k|} < 2^{|x|}$.
2. We know that the ciphertext value y_0 occurs with non-zero probability: $\Pr[y_0] > 0$.
3. We define the set of all decryptions of y_0 under all keys: $S = \{D_k(y_0) \mid k \in \mathbb{K}\}$.
4. From the size mismatch between keys and messages, it follows that there exists some message x that is not in this decryption set: $\exists k \in \mathbb{K} : x \notin S$.
5. However, if for every key $k \in \mathbb{K}$ we have that the encryption of x never produces y_0, i.e. $\forall\, k \in \mathbb{K} : E_k(x) \neq y_0$, then the probability of observing y_0 must be zero: $\Pr[y_0] = 0$

Can we have a perfect cipher with |k| < |x|?

Shannon's Theorem:
A cipher cannot be perfect if the size of its key space is less than the size of its message space.

Proof by contradiction:

1. We assume that the keyspace is smaller than the message space: $2^{|k|} < 2^{|x|}$.
2. We know that the ciphertext value y_0 occurs with non-zero probability: $\Pr[y_0] > 0$.
3. We define the set of all decryptions of y_0 under all keys: $S = \{D_k(y_0) \mid k \in \mathbb{K}\}$.
4. From the size mismatch between keys and messages, it follows that there exists some message x that is not in this decryption set: $\exists k \in \mathbb{K} : x \notin S$.
5. However, if for every key $k \in \mathbb{K}$ we have that the encryption of x never produces y_0, i.e. $\forall\, k \in \mathbb{K} : E_k(x) \neq y_0$, then the probability of observing y_0 must be zero: $\Pr[y_0] = 0$
6. This contradicts our earlier assumption.

Symmetric Ciphers

Two main approaches:

Symmetric Ciphers

Two main approaches:

1. **Stream** ciphers:
   - inspired by OTP
   - given a secret key, generate a byte of stream called *keystream* that has the same length as the message
   - encrypt/decrypt bits from x individually using a XOR operation with the keystream (as in OTP)

Symmetric Ciphers

Two main approaches:

1. **Stream** ciphers:
   - inspired by OTP
   - given a secret key, generate a byte of stream called *keystream* that has the same length as the message
   - encrypt/decrypt bits from x individually using a XOR operation with the keystream (as in OTP)

2. **Block** ciphers:
   - split the message x in blocks of fixed size
   - encrypt/decrypt each block
   - different *operation mode*: process blocks independently, chain blocks, etc.

Stream Ciphers

Stream Ciphers

Given:

- plaintext x (x_i is the i-th bit from x)
- keystream s (where $|x| == |s|$ and s_i is the i-th bit from s)

Then:

- Encryption:

$$y_i = E_{s_i} = x_i \oplus s_i = x_i + s_i \bmod 2$$

- Decryption:

$$x_i = D_{s_i} = y_i \oplus s_i = y_i + s_i \bmod 2$$

The cipher must provide the keystream generator.

Stream Ciphers

Given:

- plaintext x (x_i is the i-th bit from x)
- keystream s (where $|x| == |s|$ and s_i is the i-th bit from s)

Then:

- Encryption:

$$y_i = E_{s_i} = x_i \oplus s_i = x_i + s_i \bmod 2$$

- Decryption:

$$x_i = D_{s_i} = y_i \oplus s_i = y_i + s_i \bmod 2$$

The cipher must provide the keystream generator.

> **NOTE**: Modulo 2 is equivalent to a bitwise XOR operation.

Synchronous vs Asynchronous Stream Ciphers

A stream cipher is called:

- **synchronous** when s_i is a function of the key
  - Real-world examples: RC4, ChaCha20, Salsa20, SNOW 3G
- **asynchronous** when s_i is a function of the key and previous bits of y
  - Real-world examples: CFB-mode stream constructions

RC-4: Rivest Cipher 4

RC-4: Ron's Code (or Rivest Cipher 4)

- Designed in 1987 by Ron Rivest, trade secret until 1994 when its description was leaked
- Synchronous, variable key length, very fast to compute
- Starting from the key it will apparently generate a random stream, eventually it will repeat the sequence. However, the period is very long $(> 10^{100})$.
- Several attacks along the years:
  - From 2015, there is speculation that some state cryptologic agencies may possess the capability to break RC4 when used in the TLS protocol.
  - IETF has published RFC 7465 to prohibit the use of RC4 in TLS.
  - Mozilla and Microsoft have issued similar recommendations.

RC-4: Key Scheduling Algorithm (KSA)

```
j = 0
S = list(range(256))
for i in range(256):
    j = (j + S[i] + key[i % len(key)]) % 256
    S[i], S[j] = S[j], S[i]
```

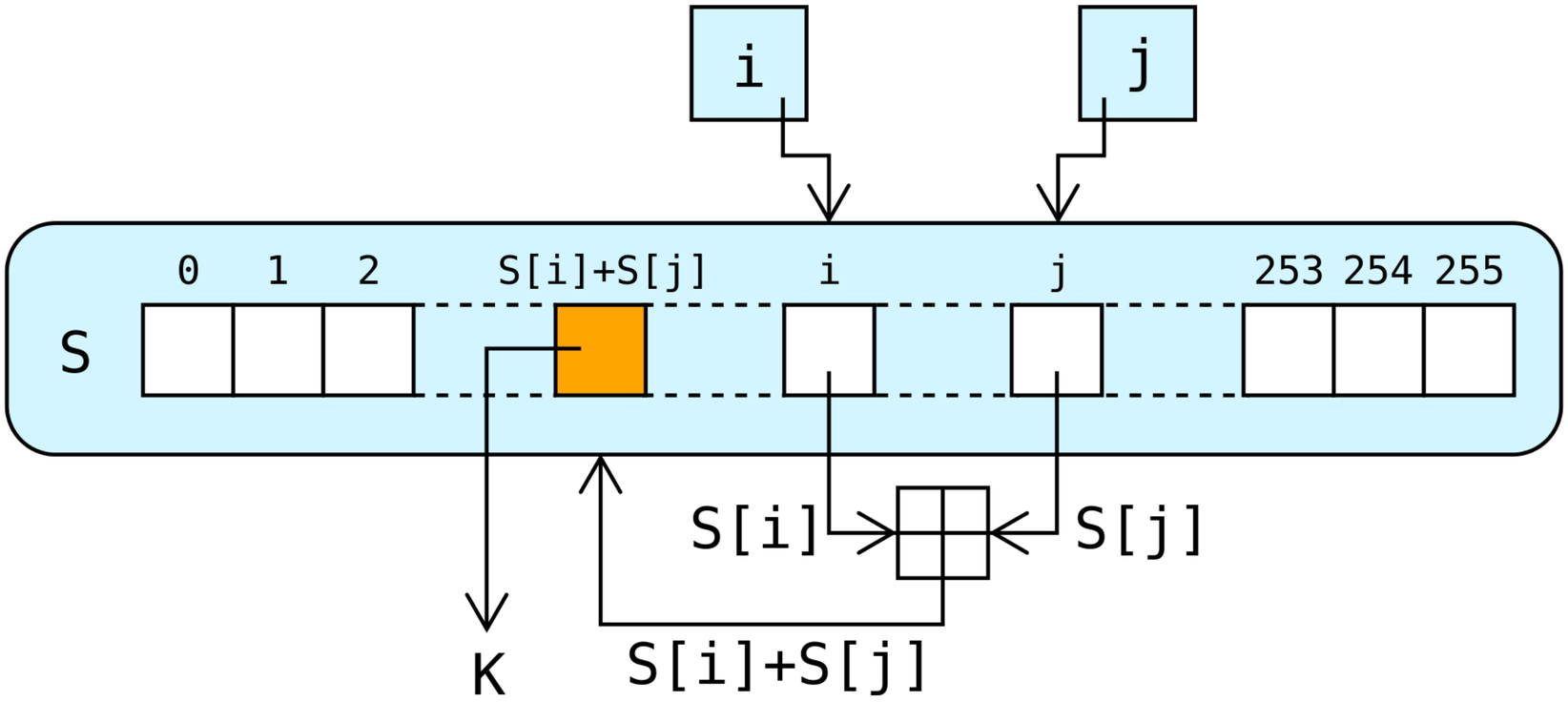
At the end S is a permutation of $\{0, \ldots, 255\}$ generated based on the value of the key.

RC-4: Pseudo-Random Generation Algorithm (PRGA)

```
i = 0
j = 0
ciphertext = []

for n in range(len(plaintext)):
    i = (i + 1) % 256        # Increment i by 1
    j = (j + S[i]) % 256     # Increment j based on S[i]
    # Swap S[i] and S[j]
    temp = S[i]
    S[i] = S[j]
    S[j] = temp
    # Generate keystream byte
    K = S[(S[i] + S[j]) % 256]
    # Encrypt
    ciphertext.append(K ^ plaintext[n])
```

Visually:



RC-4: how to encrypt different messages?

RC-4: how to encrypt different messages?

PROBLEM: We cannot use the same key for two messages, otherwise the keystream could be recovered as in OTP. Hence, we need a way of *randomizing* each keystream.

RC-4: how to encrypt different messages?

PROBLEM: We cannot use the same key for two messages, otherwise the keystream could be recovered as in OTP. Hence, we need a way of *randomizing* each keystream.

IDEA: Combine the key with an **initialization vector** (**IV**) that acts as a randomizer.

RC-4: how to encrypt different messages?

PROBLEM: We cannot use the same key for two messages, otherwise the keystream could be recovered as in OTP. Hence, we need a way of *randomizing* each keystream.

IDEA: Combine the key with an **initialization vector** (**IV**) that acts as a randomizer.

WARNING: Some algorithms are weak to attacks due to an unsecure combination of IV and the key.

RC-4: how to encrypt different messages?

PROBLEM: We cannot use the same key for two messages, otherwise the keystream could be recovered as in OTP. Hence, we need a way of *randomizing* each keystream.

IDEA: Combine the key with an **initialization vector** (**IV**) that acts as a randomizer.

WARNING: Some algorithms are weak to attacks due to an unsecure combination of IV and the key.

WEP (Wired Equivalent Privacy):

- used for WiFi in the past
- IV is 24 bits, key is 40 bits
- problem:
  - there is a 50% probability the same IV will repeat after 5,000 packets.
  - ...and the attacker is often able to *generate* traffic in a network.

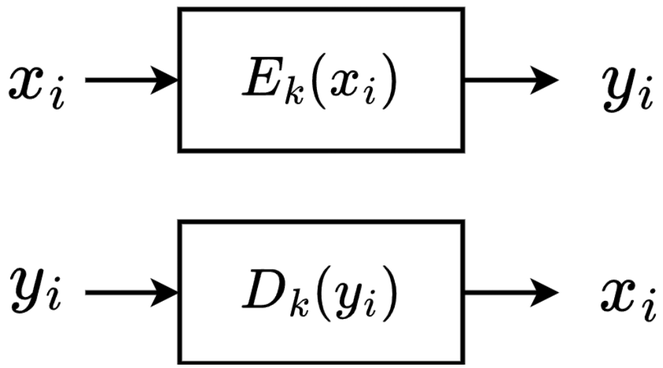
Block Ciphers

Block Ciphers

Given:

- a block x_i of the message x of h bits (h is <u>fixed</u> by the block cipher)
- a key k of n bits (n <u>fixed</u> by the block cipher)
- n can be different from h

Then:



Real-world ciphers: DES/2-DES/3-DES, IDEA, RC-2, RC-5, Blowfish, **AES**

What if the message is larger than the block size?

Strategy:

1. Divide the message into blocks of size h
2. Exploit an ***operation mode*** to compose while encrypting the blocks.

There are several operation modes:

- Electronic Code Block (ECB)
- Cipher Block Chaining (CBC)
- Propagating Cipher Block Chaining (PCBC)
- Output Feedback Mode (OFB)
- Cipher Feedback Mode (CFB)
- Counter Mode (CTR)

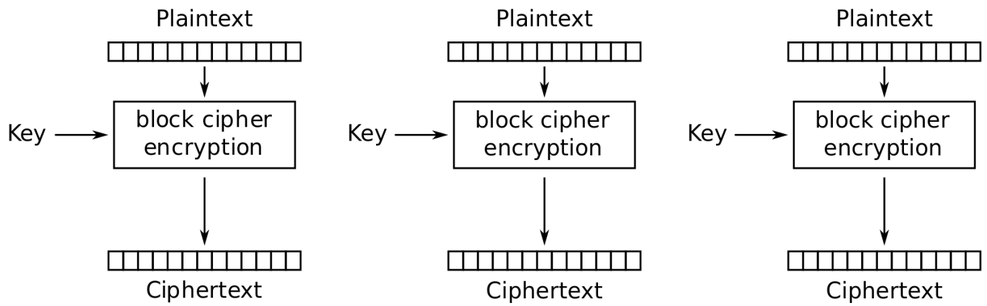
Electronic Code Block (ECB)

IDEA: Encrypt each plaintext block independently, then append the ciphertext blocks.

Electronic Code Block (ECB)

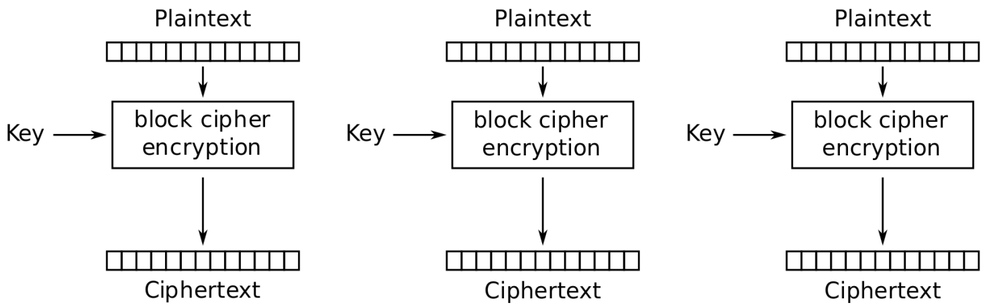
IDEA: Encrypt each plaintext block independently, then append the ciphertext blocks.

Encryption:



Electronic Codebook (ECB) mode encryption

Decryption:



Electronic Codebook (ECB) mode encryption

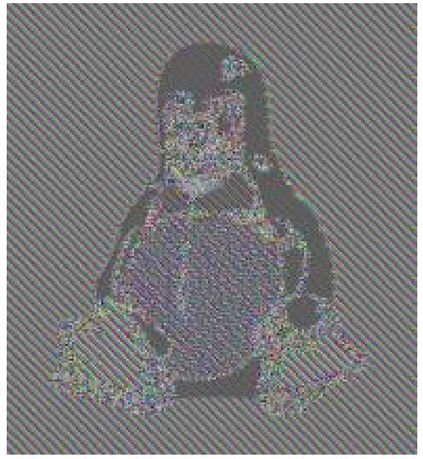
Simple and efficient (easy to parallelize)

Electronic Code Block (ECB): plaintext patterns

Plaintext

Electronic Code Block (ECB): plaintext patterns

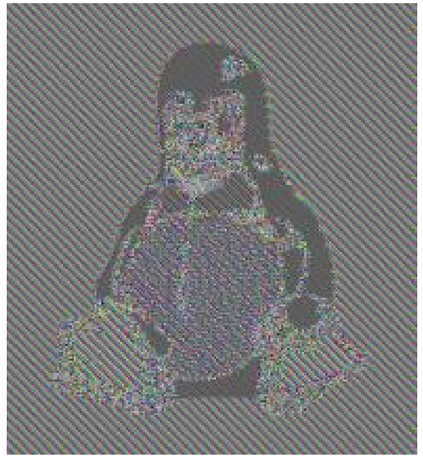
Plaintext



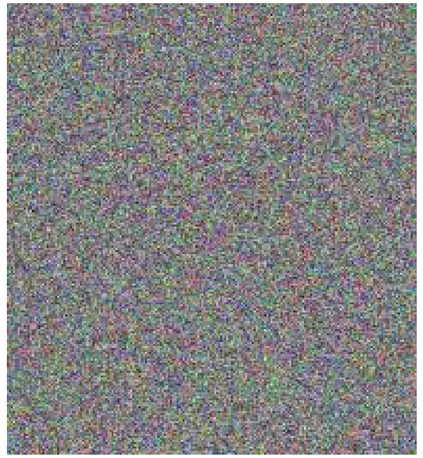
Ciphertext

Electronic Code Block (ECB): plaintext patterns

Plaintext



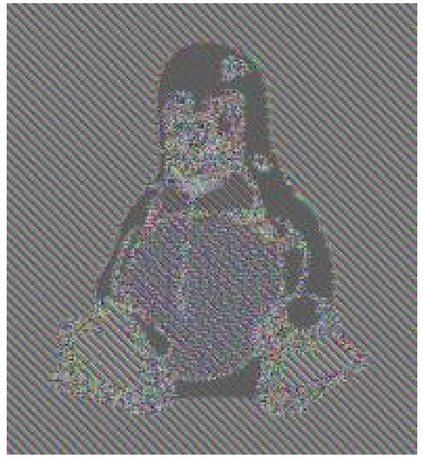
Ciphertext



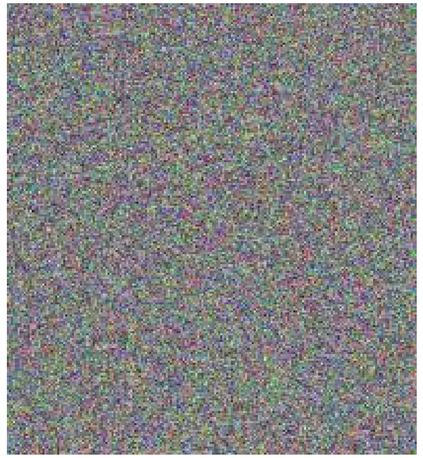
What we would like!

Electronic Code Block (ECB): plaintext patterns

Plaintext



Ciphertext



What we would like!

PROBLEM: Does not conceal plaintext patterns!

Electronic Code Block (ECB): Attacks

- **Blocks reordering**: if the message is "Bob transfers $10 to Oscar", it could be manipulated to "Oscar transfers $10 to Bob".
- **Blocks can replaced, removed, appended**: if the message is "Transfer $10 to Oscar", it could be manipulated to "Transfer $10000 to Oscar".
- **Encrypting the same message twice will generate the same ciphertext**: anyone can detect that are sending the same message twice, which could be a privacy issue.

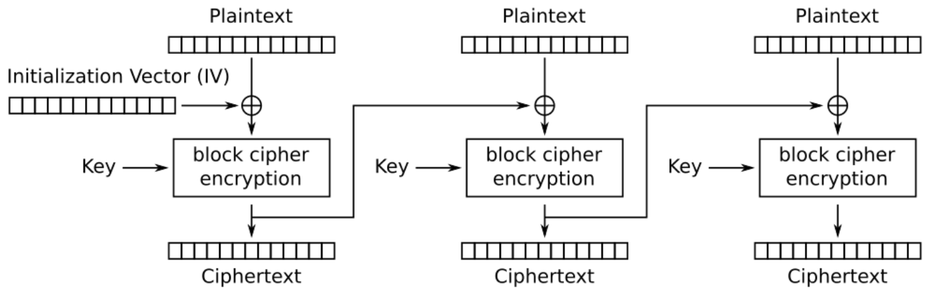
Cipher Block Chaining (CBC)

IDEA: Each plaintext block is XORed with the previous ciphertext block.

Cipher Block Chaining (CBC)

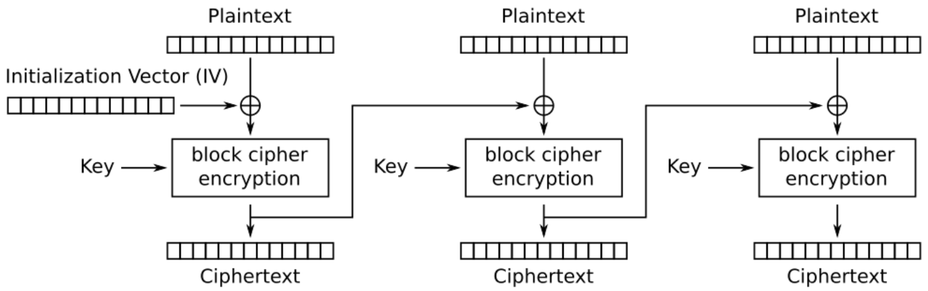
IDEA: Each plaintext block is XORed with the previous ciphertext block.

Encryption:



Cipher Block Chaining (CBC) mode encryption

Decryption:



Cipher Block Chaining (CBC) mode encryption

Notice that:

- Block y_i depends on y_{i-1}
- An IV is used at the start to randomize the process
- Parallelization: no for encryption, yes for decryption

Cipher Block Chaining (CBC): Some Details

- Impact of bit flips:
  - If one bit flipped in x_i then all subsequent blocks are affected!
  - If one bit is flipped in y_{i-1} then xi is affected an unpredictable manner, while x_i in a predictable manner. This could be exploited by an attacker. Hence we need to use integrity checks (e.g. CRC) to detect such attacks.

- Initialization Vector:
  - Must change for each message, otherwise an attacker can see when we are encrypting again the same message.
  - Can be made public, it is not a secret.

- CBC can be seen as an asynchronous stream cipher. As for ECB, we use padding to make the length of the message a multiple of the block size.

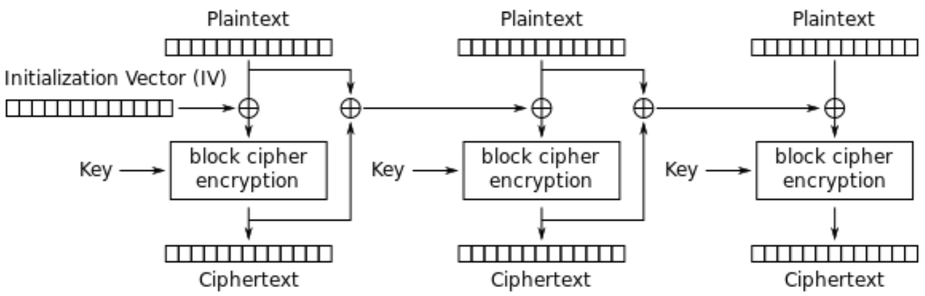
Propagating Cipher Block Chaining (PCBC)

IDEA: Propagate any (flipper or errored) bit both during encryption and decryption.

Propagating Cipher Block Chaining (PCBC)

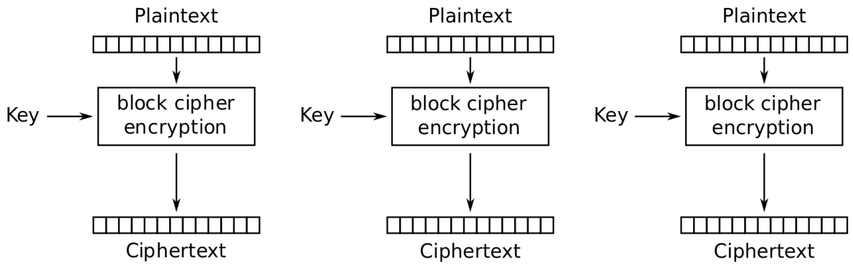
IDEA: Propagate any (flipper or errored) bit both during encryption and decryption.

Encryption:



Propagating Cipher Block Chaining (PCBC) mode encryption

Decryption:



Electronic Codebook (ECB) mode encryption

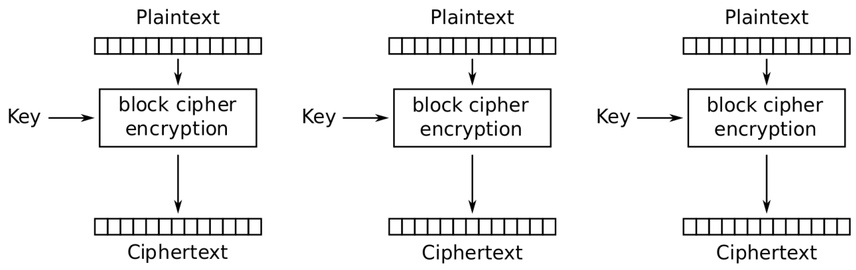
- Small changes affect all following blocks in both encryption and decryption
- Swapping two adjacent ciphertext blocks does not affect later blocks

Cipher FeedBack (CFB)

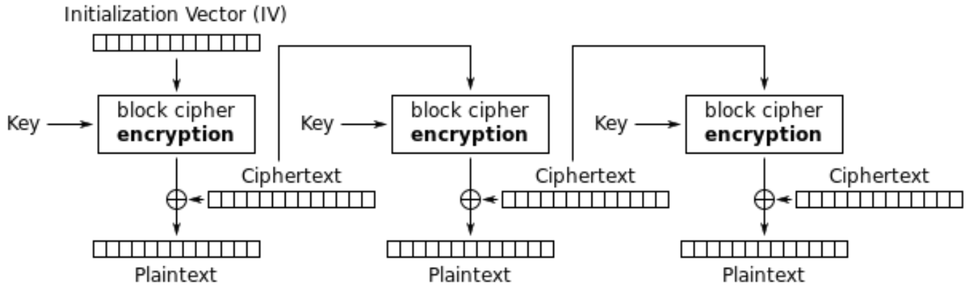
IDEA: Encrypt the previous ciphertext (or IV) to get a keystream, XOR it with the current plaintext to get the ciphertext, then feed this ciphertext back as input for the next keystream.

Cipher FeedBack (CFB)

IDEA: Encrypt the previous ciphertext (or IV) to get a keystream, XOR it with the current plaintext to get the ciphertext, then feed this ciphertext back as input for the next keystream.



Electronic Codebook (ECB) mode encryption



Cipher Feedback (CFB) mode decryption

- asynchronous stream cipher
- error propagation in encryption
- encryption is not parallelizable
- same encryption algorithm is used in both encryption and decryption

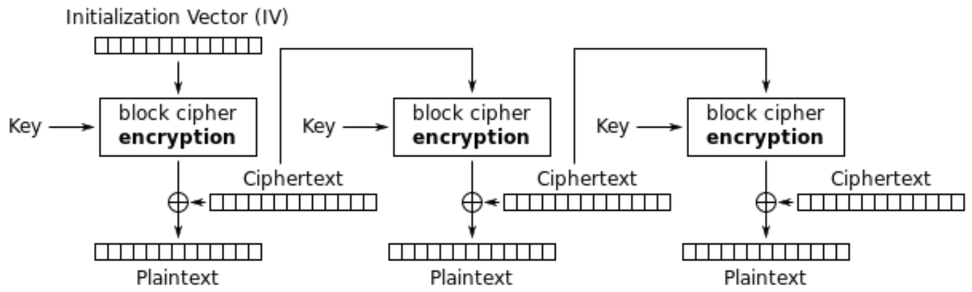
- decryption can be parallelized
- one bit error in ciphertext blocks, affect two plaintext blocks, other blocks are fine
- no need of padding

Output FeedBack (OFB)

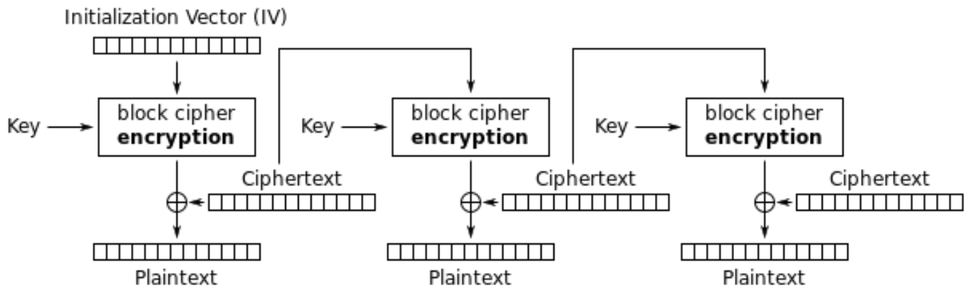
IDEA: Turn the block cipher into a synchronous stream cipher by repeatedly encrypting an IV to generate a keystream that is XORed with plaintext.

Output FeedBack (OFB)

IDEA: Turn the block cipher into a synchronous stream cipher by repeatedly encrypting an IV to generate a keystream that is XORed with plaintext.



Cipher Feedback (CFB) mode decryption



Cipher Feedback (CFB) mode decryption

- synchronous stream cipher
- one bit flip in ciphertext affect only one bit in the output (this helps using error correction codes)
- encryption and decryption are not parallelizable
- IV must be different for each message

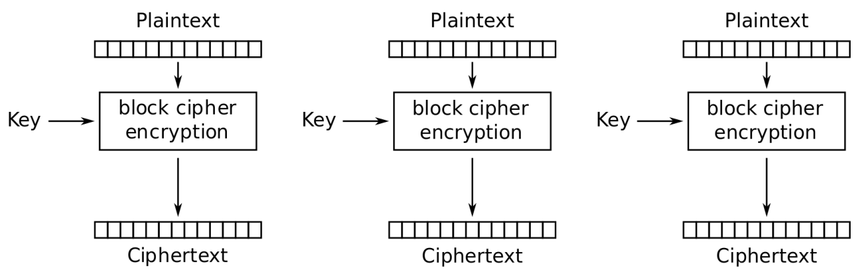
- same encryption algorithm is used both in encryption and decryption
- if encryption function and the key are known the adversary, then IV must be kept secret
- if the key is not known by the attacker, then IV can be sent in clear

Counter Mode (CTR)

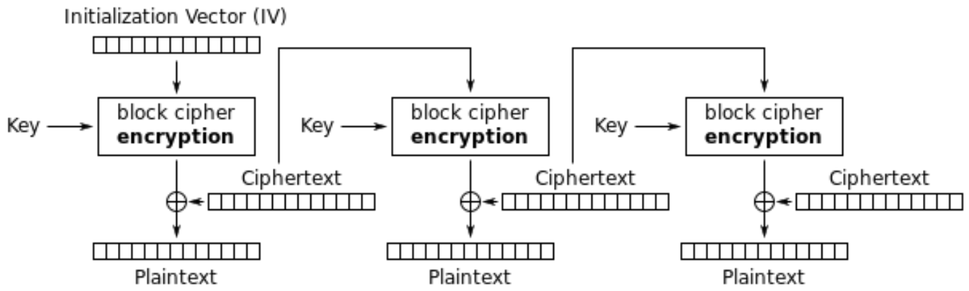
IDEA: Encrypt a nonce+counter and XOR with the plaintext to get a stream cipher.

Counter Mode (CTR)

IDEA: Encrypt a nonce+counter and XOR with the plaintext to get a stream cipher.



Electronic Codebook (ECB) mode encryption



Cipher Feedback (CFB) mode decryption

- Asynchronous stream cipher
- Besides IV (nonce), it uses
- A counter that is incremented for each block

- Encryption is parallelizable
- Decryption is parallelizable
- Same encryption algorithm is used in encryption and decryption

Operation modes: summary

	ECB	CBC	CFB	OFB	CTR
	Electronic Code Book	Cipher Block Chaining	Output Feedback	Cipher Feedback	Counter
Information leakage	High	low	low	low	low
Encryption parallelizable	Yes	No	No	No	Yes
Decryption parallelizable	Yes	Yes	Yes	No	Yes
Ciphertext manipulation	Yes	No	No	No	No
Precompute	No	No	No	Yes	Yes
Encryption error propagation	No	Yes	Yes	No	No
Decryption error propagation	No	Partial (2 Blocks)	Partial (2 Blocks)	No	No
Random access decryption	Yes	Partial	Partial	No	Yes

Table 1: Comparison of block cipher modes.

Advanced Encryption Standard (AES)

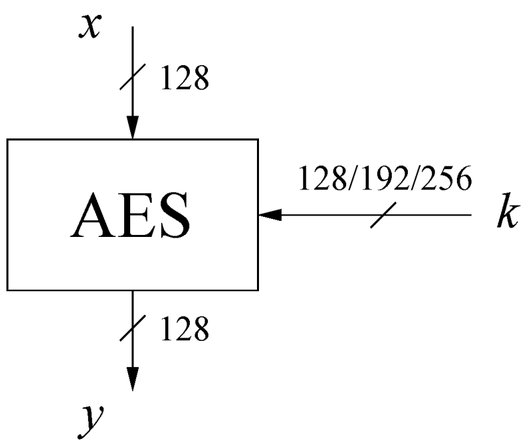
Advanced Encryption Standard (AES)

- AES is the most widely used symmetric cipher today
- The algorithm for AES was chosen by the US National Institute of Standards and Technology (NIST) in a multi-year selection process.
- The requirements for all AES candidate submissions were:
  - Block cipher with 128-bit block size
  - Three supported key lengths: 128, 192 and 256 bit
  - Security relative to other submitted algorithms
  - Efficiency in software and hardware

Chronology of the AES Selection

- Open call for a new block cipher announced by NIST in January, 1997
- 15 candidates algorithms accepted in August, 1998
- 5 finalists announced in August, 1999:
  - *Mars* – IBM Corporation
  - *RC6* – RSA Laboratories
  - *Rijndael* – J. Daemen & V. Rijmen
  - *Serpent* – Eli Biham et al.
  - *Twofish* – B. Schneier et al.
- In October 2000, Rijndael was chosen as the AES
- AES was formally approved as a US federal standard in November 2001. NSA allows to use AES with 192/256 bit key.

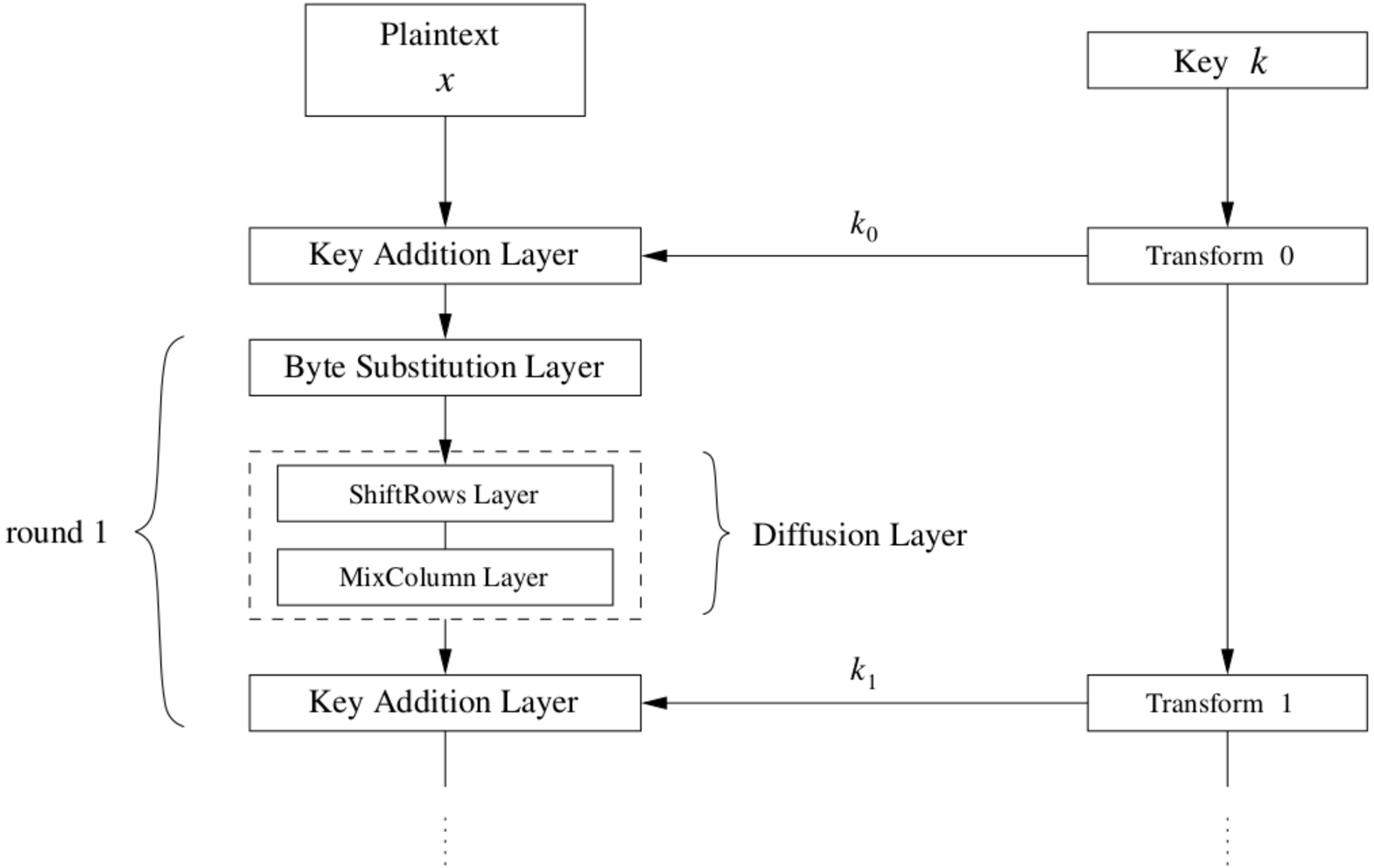
AES: Overview

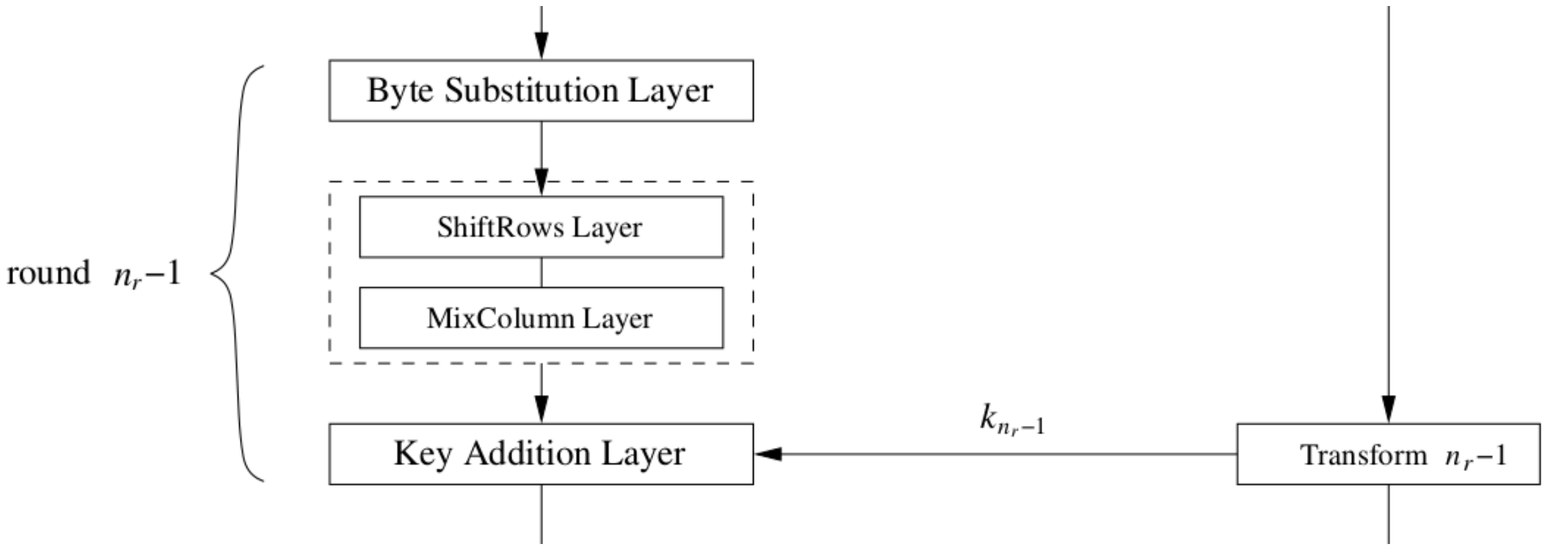


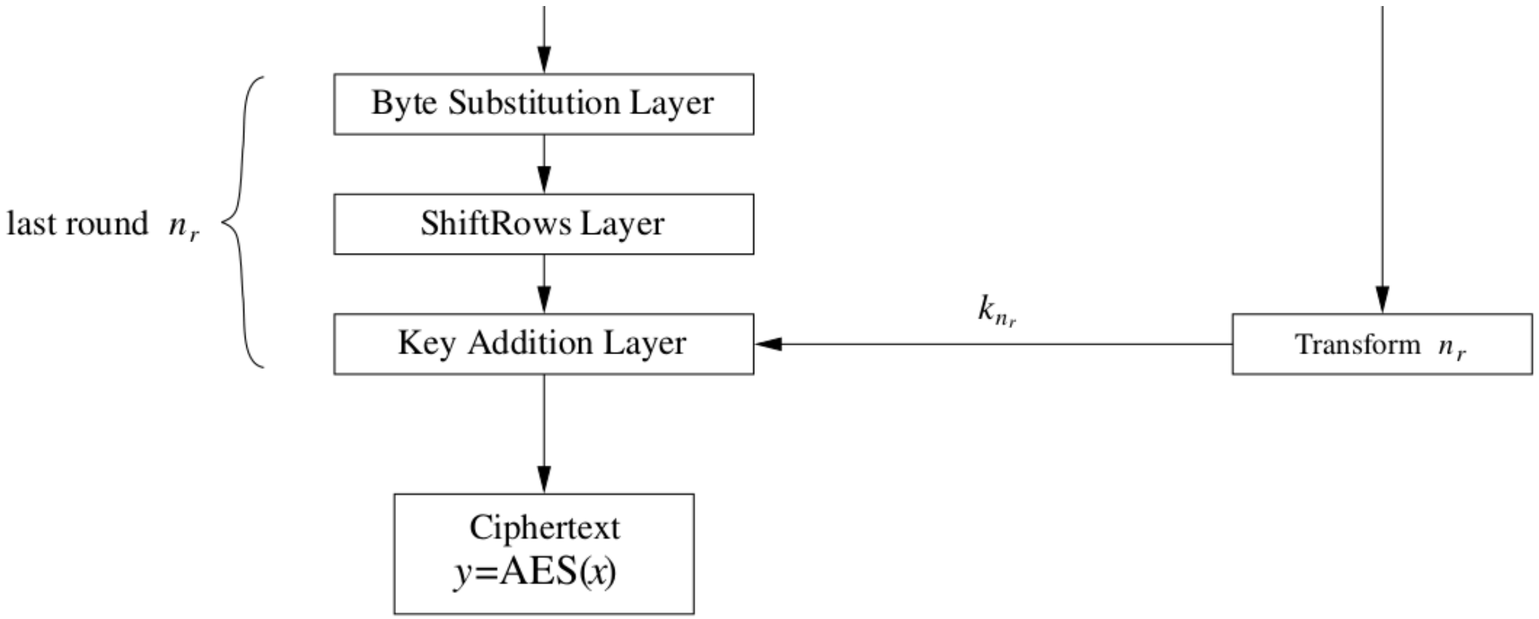
The number of rounds depends on the chosen key length:

Key length (bits)	Number of rounds
128	10
192	12
256	14

Each round consists of different “layers”:




round n_r-1
Byte Substitution Layer
ShiftRows Layer
MixColumn Layer
Key Addition Layer
k_{n_r-1}
Transform n_r-1


Byte Substitution Layer
ShiftRows Layer
Key Addition Layer
last round n_r
k_{n_r}
Transform n_r
Ciphertext
y=AES(x)

AES: Layers

Each round consists of four main layers:

1. **Byte Substitution** (goal: confusion)
2. **Shift Row** (goal: diffusion)
3. **Mix Column** (goal: diffusion)
4. **Key Addition** (goal: whitening)

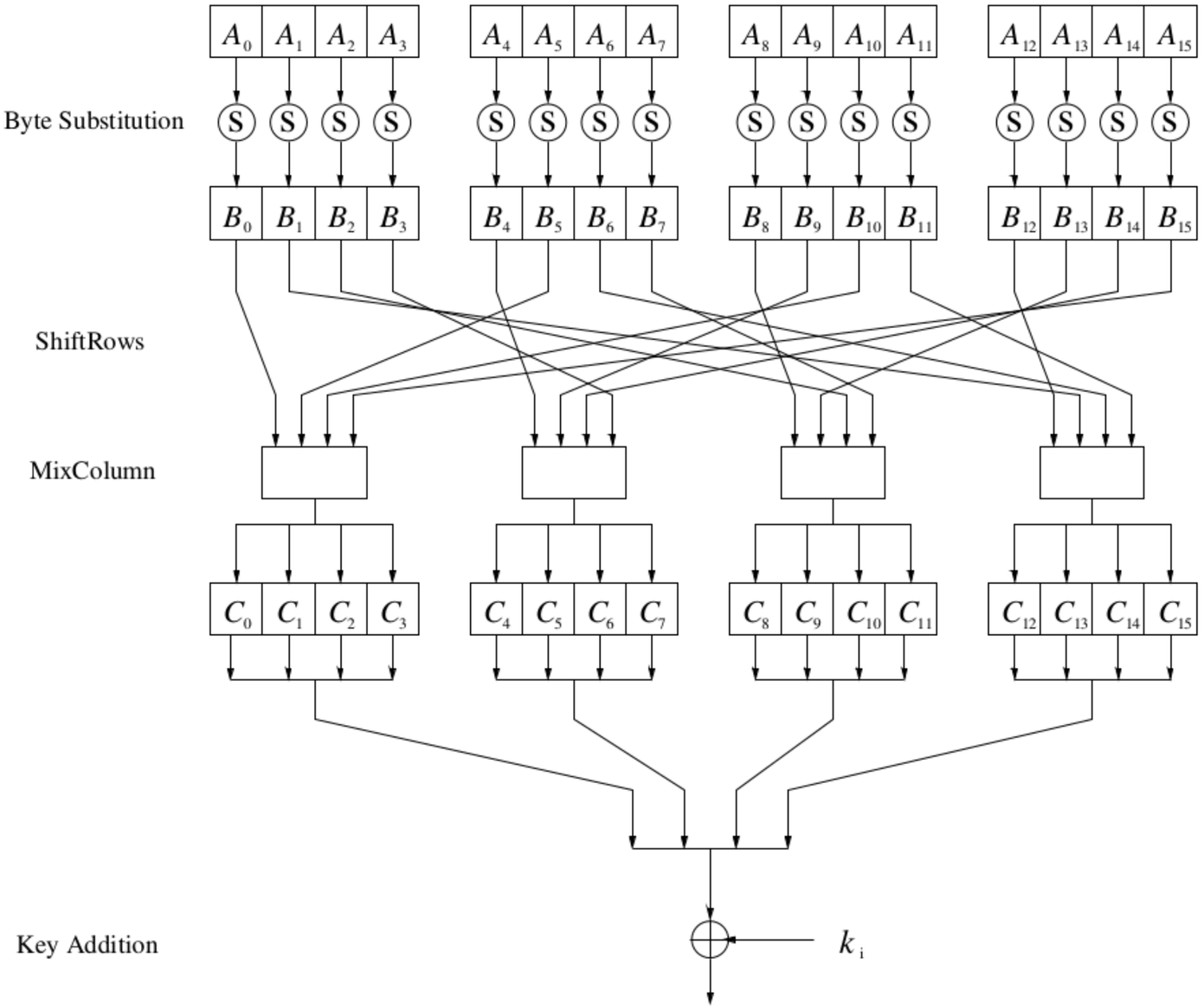
Last round does not have MixColumn layer.

Where:

- **Confusion**: make the relationship between the key and the ciphertext as complex and unpredictable as possible, so that changing the key or a plaintext bit causes non-obvious, seemingly random changes in the ciphertext (typically achieved with non-linear operations like S-boxes).

- **Diffusion**: spread the influence of each plaintext and key bit across many ciphertext bits, so that a small change in the input (e.g., flipping a single bit) results in large, seemingly random changes throughout the output; this is usually achieved with linear mixing operations that thoroughly blend bits across positions.

- **Whitening**: the process of XORing the plaintext (before the first round) and/or the intermediate state (after the last round) with one or more subkeys derived from the main key, so that the data entering and leaving the core round function is already "mixed" with secret material. This makes many classical attacks harder because an attacker never sees the raw input or output of the round function, and simple structures (like linear relations) are obscured by these extra key-dependent layers.


Byte Substitution
ShiftRows
MixColumn
Key Addition
A_0 A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8 A_9 A_{10} A_{11} A_{12} A_{13} A_{14} A_{15}
S
B_0 B_1 B_2 B_3 B_4 B_5 B_6 B_7 B_8 B_9 B_{10} B_{11} B_{12} B_{13} B_{14} B_{15}
C_0 C_1 C_2 C_3 C_4 C_5 C_6 C_7 C_8 C_9 C_{10} C_{11} C_{12} C_{13} C_{14} C_{15}
k_i

Internal Structure of AES

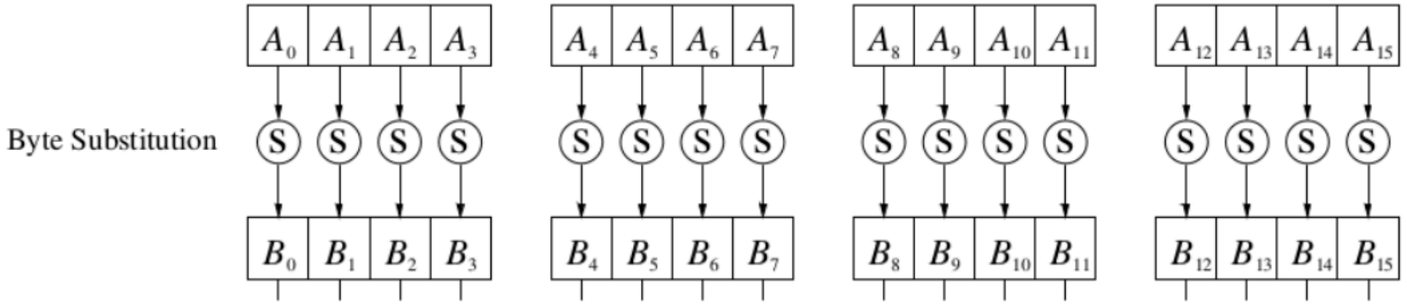
AES is a byte-oriented cipher based on a substitution-permutation network (SPN).

The state A (i.e., the 128-bit data path) can be arranged in a 4x4 matrix:

A0	A4	A8	A12
A1	A5	A9	A13
A2	A6	A10	A14
A3	A7	A11	A15

with A0, ..., A15 denoting the 16-byte input of AES

Confusion Layer



The Byte Substitution layer consists of 16 S-Boxes with the following properties:

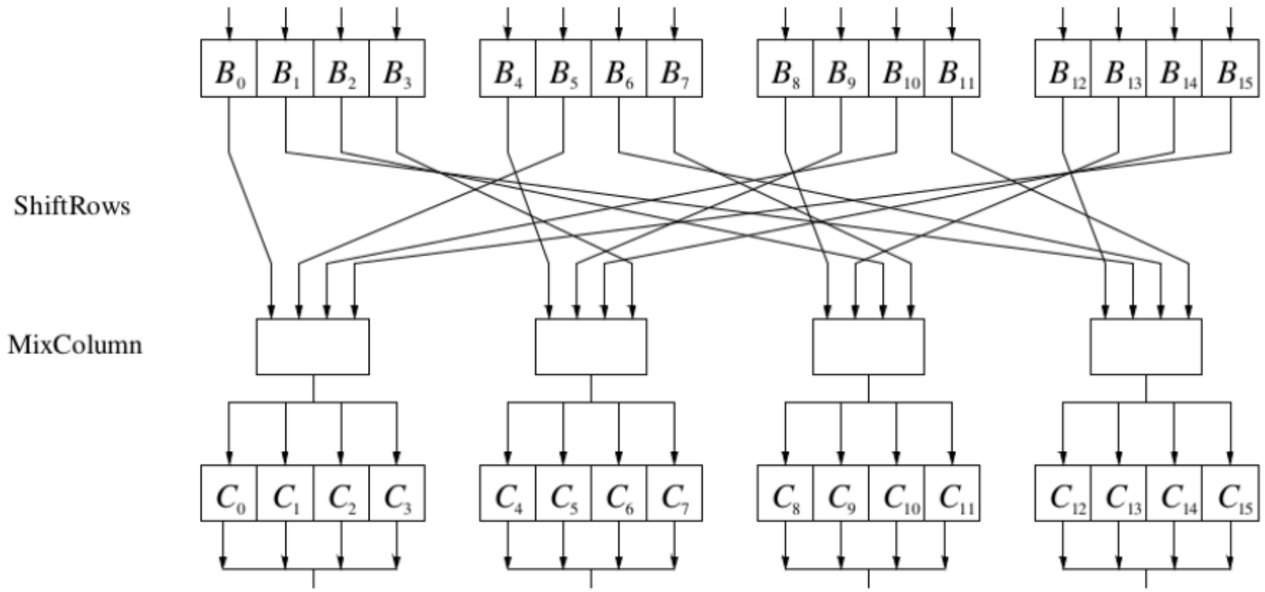
- The S-Boxes are ***identical***
- The only ***nonlinear*** elements of AES, i.e., $ByteSub(A_i) + ByteSub(A_j) \neq ByteSub(A_i + A_j)$
- ***Bijective***, i.e., there exists a one-to-one mapping of input and output bytes
  $\Rightarrow$ S-Box can be uniquely reversed
- ***Confusion***: if you flip one bit in A_i, it will affect on average 3 or 4 bits in B_i

In software implementations, the S-Box is usually realized as a lookup table.

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
x	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

$S(A_i) = B_i$ for instance: $S(\texttt{0xC2}) = \texttt{0x25}$

Diffusion Layers



- ***Diffusion***: given a byte with some bit flips, it will spread the effect on 32 bits from the state.
- consists of two sublayers:
  - ***ShiftRows*** Sublayer: Permutation of the data on a byte level
  - ***MixColumn*** Sublayer: Matrix operation which combines ("mixes") blocks of four bytes
- performs a linear operation on state matrices A, B, i.e., DIFF(A) + DIFF(B) = DIFF(A + B)

Key Addition Layer

Inputs:

- 16-byte state matrix C
- 16-byte subkey k_i

Output: $C \oplus k_i$

The different subkeys k_i are generated by the ***key schedule***, which we omit here for the sake of simplicity.

AES: Decryption

The decryption process is similar to the encryption process, but in reverse order:

1. AddRoundKey
2. InvShiftRows
3. InvSubBytes
4. InvMixColumns

All these steps are trivial given the key.

Using AES in the real world

...or at least in the Python world!

AES in Python: package and setup

Install the cryptography package:

```
pip install cryptography
```

Preliminary imports and setup:

```
from os import urandom

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend

backend = default_backend()

key = urandom(32)          # AES-256
block_size = 128           # bits (AES block size)

# message to encrypt (bytes)
plaintext = b'this is a message to encrypt'
```

AES-ECB (demo only)

ECB is deterministic on equal blocks and leaks patterns, hence **avoid** it in real systems!

```
# Padding
padder = padding.PKCS7(block_size).padder()
padded = padder.update(plaintext) + padder.finalize()

# Encrypt
cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=backend)
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded) + encryptor.finalize()

# Decrypt
decryptor = cipher.decryptor()
padded2 = decryptor.update(ciphertext) + decryptor.finalize()

# Unpadding
unpadder = padding.PKCS7(block_size).unpadder()
plaintext2 = unpadder.update(padded2) + unpadder.finalize()

assert plaintext2 == plaintext # This should be true!
```

AES-CBC

CBC requires a fresh random IV per message and padding.

```
iv = urandom(16)  # 128-bit IV

# Padding
padder = padding.PKCS7(block_size).padder()
padded = padder.update(plaintext) + padder.finalize()

# Encrypt
cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=backend)
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded) + encryptor.finalize()

# Decrypt (needs the same iv)
decryptor = cipher.decryptor()
padded2 = decryptor.update(ciphertext) + decryptor.finalize()

# Unpadding
unpadder = padding.PKCS7(block_size).unpadder()
plaintext2 = unpadder.update(padded2) + unpadder.finalize()

assert plaintext2 == plaintext # This should be true!
```

AES-CTR

CTR is a stream-like mode: no padding, but the nonce must never repeat for the same key.

```
nonce = urandom(16)  # This is the initial counter block (16 bytes)

# Encrypt
cipher = Cipher(algorithms.AES(key), modes.CTR(nonce), backend=backend)
encryptor = cipher.encryptor()
ciphertext = encryptor.update(plaintext) + encryptor.finalize()

# Decrypt
decryptor = cipher.decryptor()
plaintext2 = decryptor.update(ciphertext) + decryptor.finalize()

assert plaintext2 == plaintext # This should be true!
```

AES-CFB

CFB is stream-like: no padding. It uses an IV (never reuse IV with same key).

```
iv = urandom(16)

# Encrypt
cipher = Cipher(algorithms.AES(key), modes.CFB(iv), backend=backend)
encryptor = cipher.encryptor()
ciphertext = encryptor.update(plaintext) + encryptor.finalize()

# Decrypt
decryptor = cipher.decryptor()
plaintext2 = decryptor.update(ciphertext) + decryptor.finalize()

assert plaintext2 == plaintext # This should be true!
```

AES-OFB

OFB is stream-like: no padding. It uses an IV (never reuse IV with same key).

```
iv = urandom(16)

# Encrypt
cipher = Cipher(algorithms.AES(key), modes.OFB(iv), backend=backend)
encryptor = cipher.encryptor()
ciphertext = encryptor.update(plaintext) + encryptor.finalize()

# Decrypt
decryptor = cipher.decryptor()
plaintext2 = decryptor.update(ciphertext) + decryptor.finalize()

assert plaintext2 == plaintext # This should be true!
```